Two Layer Fully Connected Neural Network

Optimisation in Machine Learning

Abukar Hassan

abukarab@student.chalmers.se

April 19, 2026

1 Preface

This report describes the manual implementation of a two-layer fully connected neural network. The objective was to implement all forward- and backward-pass computations without automatic differentiation, thereby ensuring full analytical control over the learning dynamics. Motivated by experimental exploration, the baseline model was extended to incorporate mini-batch optimisation and early stopping in order to improve training stability and generalisation performance. Although dropout regularisation was implemented in the forward pass, it was ultimately disabled in the final configuration, as it did not yield consistent performance improvements under the selected hyperparameter regime.

2 Two-Layer Fully Connected Neural Network

2.1 Network Architecture

The model is defined by the following architectural components:

- Input dimensionality: $d_{in} = 256$
- Hidden layer dimensionality: $d_h = 512$
- Output dimensionality: $d_{out} = 1$
- Activation function: Rectified Linear Unit (ReLU)
- Dropout probability (optional): $p = 0.5$

The forward mapping from input $x_i \in \mathbb{R}^{256}$ to prediction $\hat{y}_i \in \mathbb{R}$ is given by:

$$\hat{y}_i = W_2\, \sigma(W_1 x_i + b_1) + b_2, \tag{1}$$

where $\sigma(\cdot)$ denotes the ReLU activation function.

2.2 Forward Pass

Let $X \in \mathbb{R}^{B\times 256}$ denote a mini-batch of size B. The first affine transformation is given by:

$$Z_1 = XW_1^\top + b_1, \tag{2}$$

where $W_1 \in \mathbb{R}^{512\times 256}$ and $b_1 \in \mathbb{R}^{512}$.

The hidden-layer activations are obtained via the ReLU non-linearity:

$$A_1 = \max(0, Z_1). \tag{3}$$

When dropout is enabled during training, inverted dropout is applied to the activations:

$$\tilde{A}_1 = \frac{A_1 \odot M}{1-p}, \quad M \sim \text{Bernoulli}(1-p), \tag{4}$$

where $\odot$ denotes element-wise multiplication.

The output layer then computes:

$$\hat{y} = \tilde{A}_1 W_2^\top + b_2, \tag{5}$$

with $W_2 \in \mathbb{R}^{1\times 512}$ and $b_2 \in \mathbb{R}$.

2.3 Loss Function

The optimisation objective is the Mean Squared Error (MSE), defined over a batch of B samples:

$$\mathcal{L} = \frac{1}{B}\sum_{i=1}^{B}(\hat{y}_i - y_i)^2. \tag{6}$$

2.4 Optimisation via Gradient Descent

Parameter updates are performed using Stochastic Gradient Descent (SGD), following:

$$\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}, \tag{7}$$

where η denotes the learning rate.

The training configuration is summarised as:

- Learning rate: $\eta = 10^{-4}$
- Batch size: $B = 128$
- Early stopping patience: 100 epochs

Mini-batch optimisation improves convergence stability by reducing gradient variance, whilst early stopping mitigates overfitting by restoring parameters corresponding to the best validation performance.

2.5 Backpropagation

The gradient of the loss with respect to the predictions is:

$$\frac{\partial \mathcal{L}}{\partial \hat{y}_i} = \frac{2}{B}(\hat{y}_i - y_i). \tag{8}$$

Output layer gradients. The gradients for the second layer parameters are:

$$\frac{\partial \mathcal{L}}{\partial W_2} = \frac{2}{B}\sum_{i=1}^{B}(\hat{y}_i - y_i)A_{1,i}^{\top}, \tag{9}$$

$$\frac{\partial \mathcal{L}}{\partial b_2} = \frac{2}{B}\sum_{i=1}^{B}(\hat{y}_i - y_i). \tag{10}$$

Hidden layer backpropagation. The error is propagated to the hidden layer as:

$$\frac{\partial \mathcal{L}}{\partial A_1} = (\hat{y} - y)W_2. \tag{11}$$

ReLU derivative. The derivative of the ReLU activation function is defined element-wise as:

$$\frac{\partial A_1}{\partial Z_1} = \begin{cases} 1, & Z_1 > 0, \\ 0, & Z_1 \leq 0. \end{cases} \tag{12}$$

If dropout is used, the same binary mask M is applied during backpropagation to ensure consistency of the computational graph and correct gradient flow.

3 Results

Figure 1 illustrates the evolution of training and test mean squared error over 6000 epochs. The training loss decreases smoothly and stabilises after an initial transient phase. The test loss follows a similar trajectory without significant divergence, suggesting limited overfitting under the chosen

configuration.

The optimisation appears to converge relatively early, after which the loss plateaus. This behaviour is consistent with stochastic gradient descent operating in a regime where further improvements are constrained by mini-batch noise and the learning rate magnitude. Notably, dropout was not included in the final experiment; thus, the observed dynamics correspond to standard SGD with batch size 128 and learning rate 10^{-4}.

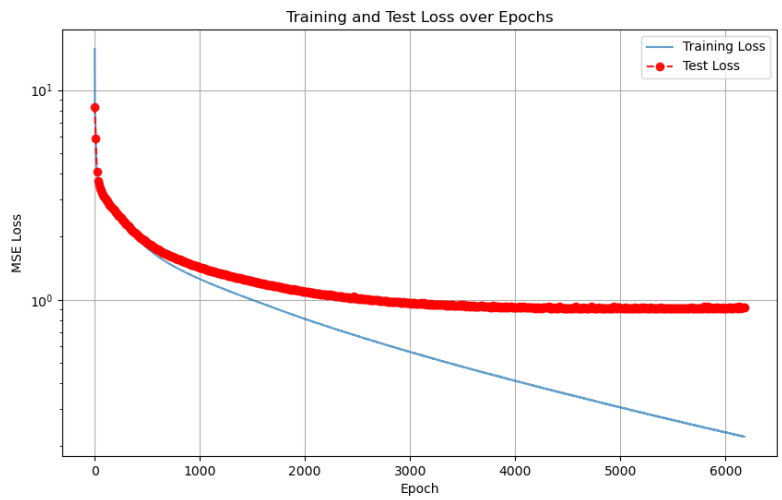

Figure 1: Training and test loss over 6000 epochs (batch size 128, learning rate 10^{-4}).

4 Conclusion

Given the presence of stochastic gradient noise induced by mini-batching, the relatively small learning rate (10^{-4}) is essential to ensure stable convergence; higher values were observed to induce oscillatory or divergent behaviour. In hindsight, the early stopping patience parameter may be considered conservative, as convergence on the validation set occurred earlier than the termination point.

An interesting observation is that while training loss continues to decrease gradually, the test loss saturates rather than diverging. This may indicate either mild overfitting controlled by optimisation noise or the presence of inherent label noise limiting achievable generalisation. Further investigation into the loss landscape, including the presence of abrupt optimisation transitions reported in similar regression settings, could provide additional insight.

Future work may explore explicit regularisation via weight decay, alternative hidden layer widths,

and momentum-based optimisers such as SGD with momentum or Adam to improve convergence efficiency and generalisation behaviour.